

Sudoku - Final Project

Extended Abstract

Fundamentals of Logic Programming
Benemérita Universidad Autónoma de Puebla
November 2025

Valeria Elizabeth Rojo
Hernández
Information Technology Engineering
rh202354373@alm.buap.mx

202354373

Erick Alejandro Rosales Rodrigo
Information Technology Engineering
rr202354929@alm.buap.mx

202354929

Ana Karen Garcia Flores
Information Technology Engineering
gf202326818@alm.buap.mx

202326818

ABSTRACT

This project presents the development of a dual-solver Sudoku system capable of resolving both traditional and irregular grids through Answer Set Programming. The system integrates two logic-based engines, Clingo and DLV, and provides a user-friendly interface implemented in Python. By separating declarative reasoning from interface interaction, the project demonstrates an efficient, scalable, and modular approach to constraint-based puzzle solving. The final product highlights the strengths of declarative paradigms for representing logical constraints while offering an accessible environment for end users.

CCS CONCEPTS

- **Theory of computation** → **Constraint and logic programming**
- **Software and its engineering** → **Logic programming languages**
- **Computing methodologies** → **Knowledge representation and reasoning**
- **Human-centered computing** → **Graphical user interfaces**

KEYWORDS

Sudoku; Clingo; DLV; Python; Graphic interface; Declarative Programming; Irregular Sudoku; Logic Reasoning; Naked pairs.

ACM Reference format:

G. Gubbiotti, P. Malagò, S. Fin, S. Tacchi, L. Giovannini, D. Bisero, M. Madami, and G. Carlotti. 1997. SIG Proceedings Paper in word Format. In *Proceedings of ACM Woodstock conference, El Paso, Texas USA, July 1997 (WOODSTOCK'97)*, 4 pages. <https://doi.org/10.1145/1234>

1 INTRODUCTION

Sudoku is a globally recognized logic puzzle that challenges players to complete a numerical grid based on simple yet strict constraints: each number must appear exactly once in every row, column, and region. While traditionally solved by hand, Sudoku provides an ideal environment for testing computational reasoning strategies due to its structured constraints.

This project explores the potential of **declarative programming** as a method for solving both traditional and irregular Sudoku layouts. Instead of implementing procedural algorithms, we rely on **Answer Set Programming (ASP)**—a paradigm in which the programmer defines rules and constraints, and the solver infers valid solutions. Two ASP engines, **Clingo** and **DLV**, were integrated into a Python-based system that manages user interaction, solver selection, and solution visualization.



Illustration 1. Modern Sudoku

The goal of this work is to demonstrate how constraint satisfaction problems can be modeled elegantly through ASP, while also providing a practical, interactive interface for users. Through this combination of logic reasoning and intuitive design, the project reveals a scalable and flexible approach to puzzle solving that can be extended to more complex variants in the future.

2 Problem definition

The project deals with two variants of Sudoku:

Traditional Sudoku.
A 9×9 grid is used. Each row, each column, and each 3×3 sub-grid must contain all digits from 1 to 9 without repetition. The game starts with a set of fixed cells (clues), and the goal is to complete the remaining cells while respecting these constraints.

Irregular Sudoku.
The grid is still 9×9, but the regions are no longer regular 3×3 blocks. Instead, cells are grouped into irregularly shaped regions. Each region must also contain all digits from 1 to 9 without repetition. In the interface, these regions are displayed with different background colors.

In both cases, the problem is formulated as a **constraint satisfaction problem**: given the initial clues and the partition of the board into regions, the task is to find an assignment of numbers to cells that satisfies all rules.

3 Methodology

3.1 System Architecture

The system architecture is divided into four main components:

1. **Graphical user interface (GUI) in Python.**
Implemented with Tkinter. It provides controls to select board size (9×9 or 4×4), Sudoku type (traditional or irregular), difficulty level, and buttons to generate games, clear the board, request hints, and solve with Clingo or DLV.
2. **Sudoku instance generation.**
Based on the user's choices, the system generates a fully solved board using the ASP model and then hides some cells to create the puzzle shown to the user. The full solution is stored internally for checking and for the hint function.
3. **Logical modules in ASP.**
A set of programs that describe the Sudoku rules in a declarative way, both for Clingo (*.lp) and DLV (*.dl). They define row, column, and region constraints, as well as how initial clues are interpreted.
4. **Solver integration layer.**
Python code that builds the facts corresponding to the current board, calls Clingo or DLV depending on the user's choice, and processes the output to update the GUI.

3.2 Sudoku Modeling in ASP

Sudoku is modeled in ASP in a generic way for an $n \times n$ board (with $n = 9$ or $n = 4$ in the implementation):

- **Domain of values and cells.**
Values 1..n and positions for rows and columns are defined.
- **Assignment of numbers to cells.**
Each cell (row, column) must take exactly one value.

- **Row and column constraints.**
Rules forbid the same value from appearing twice in the same row or column.
- **Region constraints.**
Regions (either 3×3 sub-grids in the traditional case or irregular regions) are expressed by facts indicating which region each cell belongs to. Additional rules prevent repeating a value within the same region.
- **Initial clues.**
Fixed cells are represented as input facts that must be satisfied by any solution.

The **Naked Pairs** strategy, as described in Sudoku literature, was used as a conceptual reference to understand how candidates can be eliminated inside a unit, although the ASP model itself performs the full search.

3.3 Solver Integration (Clingo and DLV)

The system offers two ways to solve the same Sudoku:

- **Clingo.**
The Python library is used. The program loads the ASP modules, adds facts for the current board state (clues and regions), and asks Clingo to compute a model. The solution is obtained as cell(row,column,value) literals and written back into the grid.
- **DLV.**
DLV is invoked as an external process. Python generates an input file with the board facts and runs the DLV command with the corresponding .dl programs. The output is parsed to reconstruct the value of each cell.

Both routes share the same general flow: generate facts from what the user sees, call the solver, and reflect the solution in the grid. This makes it possible to visually compare Clingo and DLV on the same puzzle.

3.4 Python Interface

The graphical interface is the user's main interaction point. Its main elements are:

- Configuration panel (left side).
 - Board size selector (9×9 or 4×4).
 - Game type selector: Traditional or Irregular (Color).
 - Difficulty selector.
 - Buttons:
 - Generate New Game.
 - Clear Board.
 - Solve with CLINGO.
 - Solve with DLV.
 - Give me a hint.

- Board area.

A grid of text entries represents each cell of the Sudoku. In irregular mode, background colors indicate different regions; in traditional mode, the board uses neutral colors.

Now we place the first four figures:



Illustration 2. Traditional 9x9 Sudoku interface.

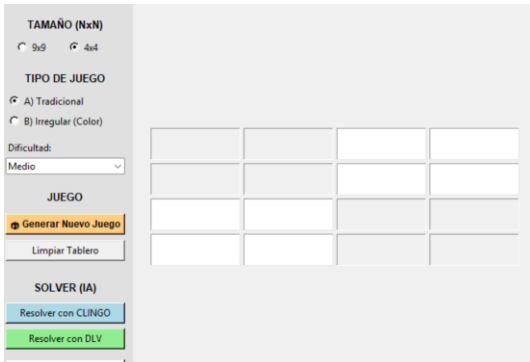


Illustration 3. Traditional 4x4 Sudoku interface.



Illustration 4. Irregular 9x9 Sudoku interface.

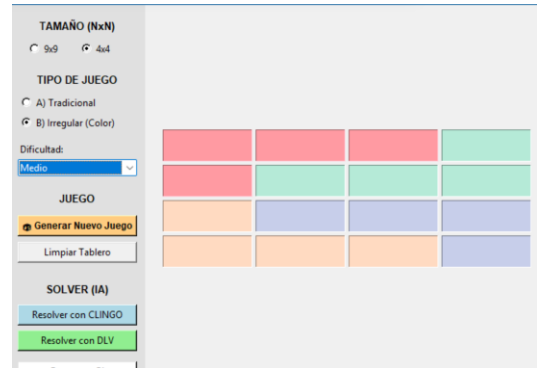


Illustration 5. Irregular 4x4 Sudoku interface.

4 Implementation

4.1 Traditional Sudoku Solver

In traditional mode, the implementation flow is:

1. **Puzzle generation.**

When *Generate New Game* is pressed, the system creates a fully solved board using the traditional Sudoku ASP model and then hides a subset of cells according to the selected difficulty. The visible clues are written into the GUI grid.

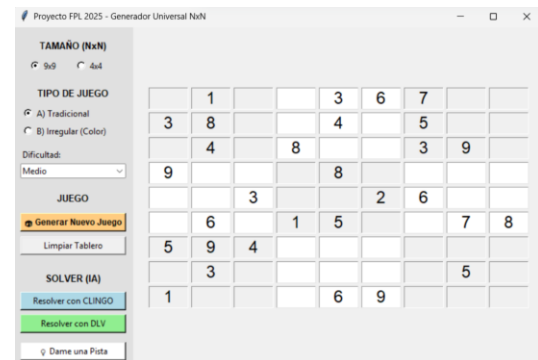


Illustration 6. Generated traditional Sudoku (medium).

2. Board clearing.

The *Clear Board* button removes all values from the grid and leaves an empty board ready for a new instance.

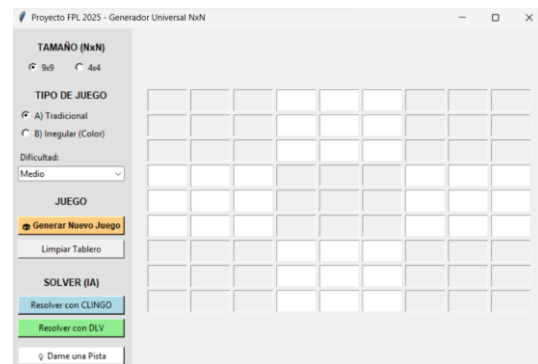


Illustration 7. Cleared traditional Sudoku board.

3. Solving with Clingo and DLV.

The user can ask the system to complete the Sudoku with either solver. The interface builds the facts for the current state, calls Clingo or DLV, and writes the solved values into the grid (typically in blue to distinguish them from the original clues).

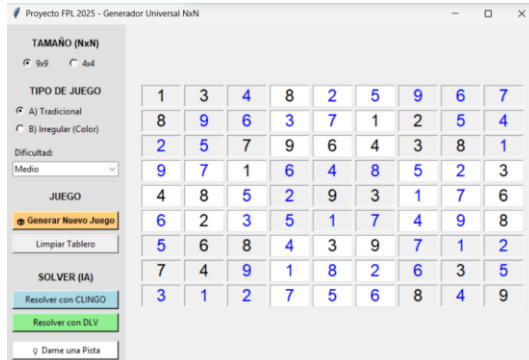


Illustration 8. Traditional Sudoku solved with Clingo.

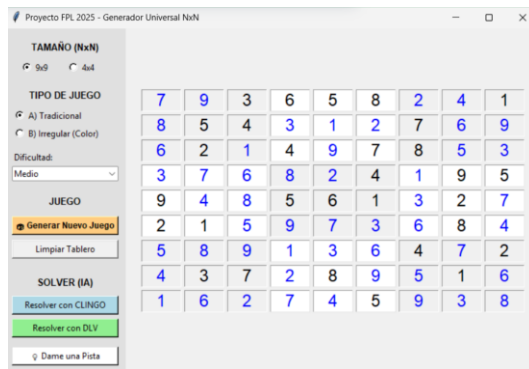


Illustration 9. Traditional Sudoku solved with DLV.

4. Hint button.

The *Give me a hint* button uses the internally stored full solution to fill one empty cell with the correct value, helping the user without revealing the entire solution.

4.2 Irregular Sudoku Solver

Irregular mode reuses most of the logic but changes the region definition:

1. **Irregular region maps.**
The region generation function creates different maps where each cell belongs to an irregularly shaped region. These maps are translated into input facts for the ASP model.
2. **Generation and clearing of irregular boards.**
Generate New Game builds a fully solved irregular Sudoku and hides values to create the puzzle; *Clear Board* erases the cells and leaves only the colored region structure.

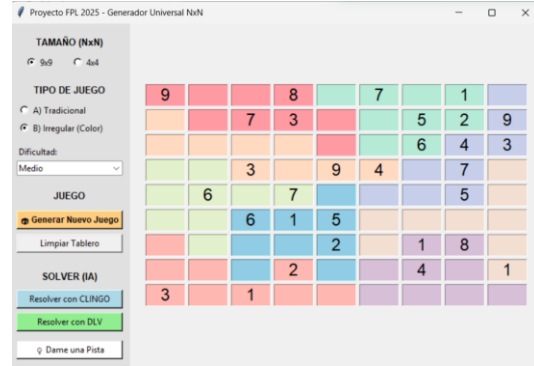


Illustration 10. Generated irregular Sudoku.

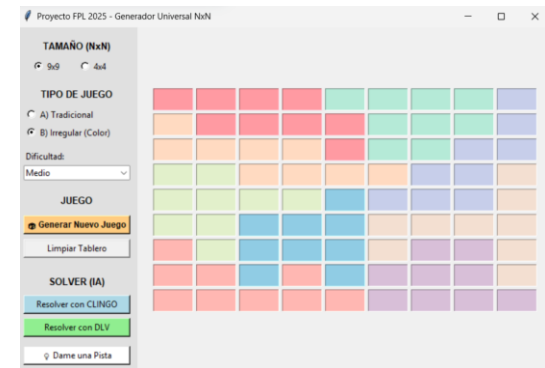


Illustration 11. Cleared irregular Sudoku board.

Just like in the traditional case, the interface can send the irregular board to Clingo or DLV. Region constraints in the ASP programs are now based on the irregular maps, but the integration flow is the same.

5 Results and Demonstration

During testing and demonstration, the following was observed:

- The system correctly generates traditional and irregular Sudokus at sizes 9×9 and 4×4.
- Clingo and DLV produce consistent solutions for the test instances.
- Changes in game type, size, and difficulty are correctly reflected in the interface.
- The hint button returns values consistent with the full solution generated by the solver.

To illustrate irregular Sudokus at different difficulty levels, the last two figures are included:



Illustration 12. Irregular Sudoku (medium) solved.



Illustration 13. Irregular Sudoku (hard) solved.

These figures show that the system can handle more challenging irregular puzzles while maintaining solution correctness.

6 Conclusions

The system demonstrates that Answer Set Programming is an effective tool to model and solve both traditional and irregular Sudokus. Integrating Clingo and DLV into a single application makes it possible to compare two different ASP solvers using the same declarative model and user interface.

The chosen architecture, which separates the GUI from the logical modules, makes the system easier to extend: new Sudoku variants or new region maps can be added mainly by modifying the input facts.

As future work, the system could be enriched with additional logical techniques, more refined criteria to control puzzle difficulty, and step-by-step explanations to help users follow the solver's reasoning.

A HEADINGS IN APPENDICES

The rules about hierarchical headings discussed above for the body of the article are different in the appendices. In the appendix environment, the command section is used to indicate the start of each Appendix, with alphabetic order designation (i.e., the first is A, the second B, etc.) and a title (if you include one). So, if you need hierarchical structure within an Appendix, start with subsection as the highest level. Here is an outline of the body of this document in Appendix-appropriate form:

A.1 Introduction

A.2 Problem definition

A.3 Methodology

A.3.1 System Architecture

A.3.2 Sudoku Modeling in ASP

A.3.3 Solver Integration (Clingo and DLV)

A.3.4 Python Interface

A.4 Implementation

A.4.1 Traditional Sudoku Solver

A.4.2 Irregular Sudoku Solver

A.5 Results and Demonstration

A.6 Conclusions

A.7 References

REFERENCES

- [1] SudokuWiki.org. n.d. *Naked Candidates – Naked Pairs*. https://www.sudokuwiki.org/Naked_Candidates#NP (accessed Nov 20 2025).
- [2] Potassco. 2024. *clingo | Potassco Project for ASP*. <https://potassco.org/clingo/> (accessed Nov 20 2025).
- [3] Potassco. 2024. *Potassco Guide to Answer Set Programming*. <https://potassco.org/guide/> (accessed Nov 20 2025).
- [4] DLV System. 2024. *DLV: A Disjunctive Datalog System for Knowledge Representation and Reasoning*. <https://www.dlvsystem.com/> (accessed Nov 20 2025).
- [5] Potassco Team. 2023. *Answer Set Solving in Practice (Online Edition)*. <https://potassco.org/book/> (accessed Nov 20 2025).